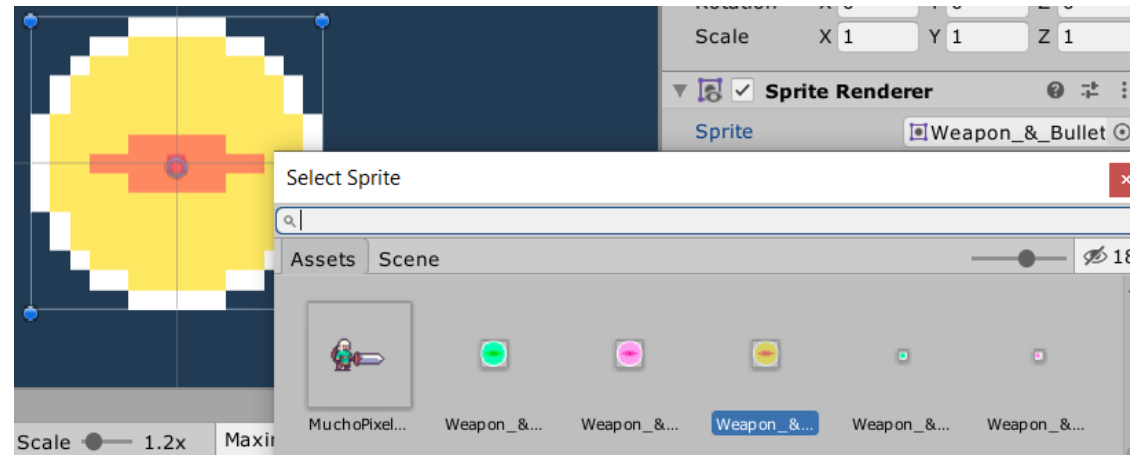


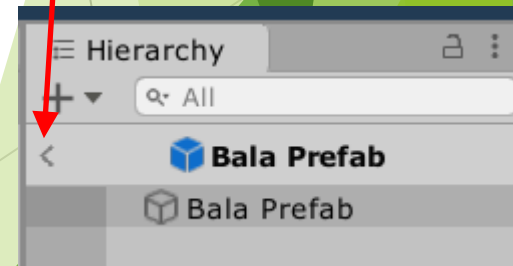
Prática 3 - Controlador do Inimigo

- ▶ Iniciaremos alterando a bala. Para isso, selecionamos o Bala Prefab na janela Project e, na janela Inspector, clicamos no botão [Open Prefab].
- ▶ Entraremos numa janela de edição, fora da cena, e poderemos mudar seu tamanho e outras características. Usando a ferramenta de escala, diminua um pouco o tamanho da bala. Mude o Sprite do Sprite Renderer, como vemos na figura abaixo:



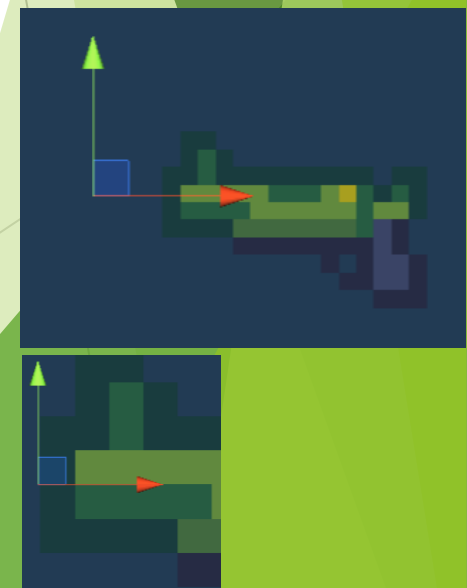
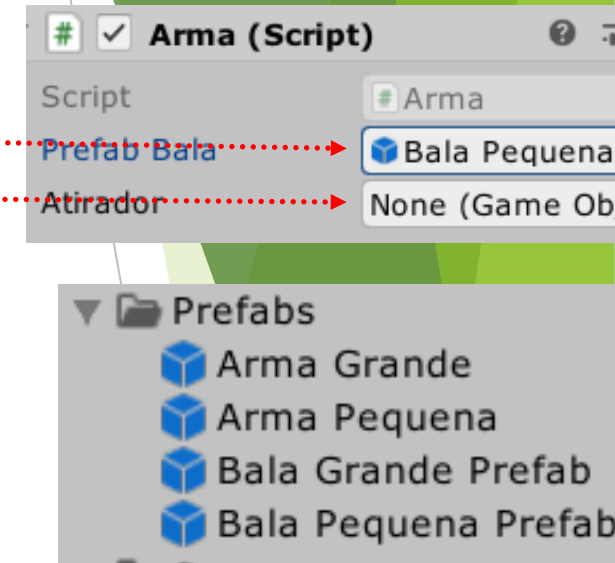
Clique aqui para
sair do editor
de Prefabs

- ▶ Note que, no canto superior direito da janela de edição do Prefab, existe um checkbox Auto Save marcado, de forma que toda alteração que fizermos será salva.
- ▶ Agora, saia do editor de prefab e mude o seu nome para “Bala Grande Prefab”. No script Bala, mude a velocidade para 2.25.
- ▶ Tendo ele selecionado, pressione [Ctrl-D] para duplicá-lo. Mude o nome para “Bala Pequena Prefab” e altere a sua velocidade, no script Bala, para 1.75. Mude o Sprite do Sprite Renderer desse novo prefab para um pequeno circulo amarelo que já temos no seletor de Sprites, proveniente do Sprite Atlas.



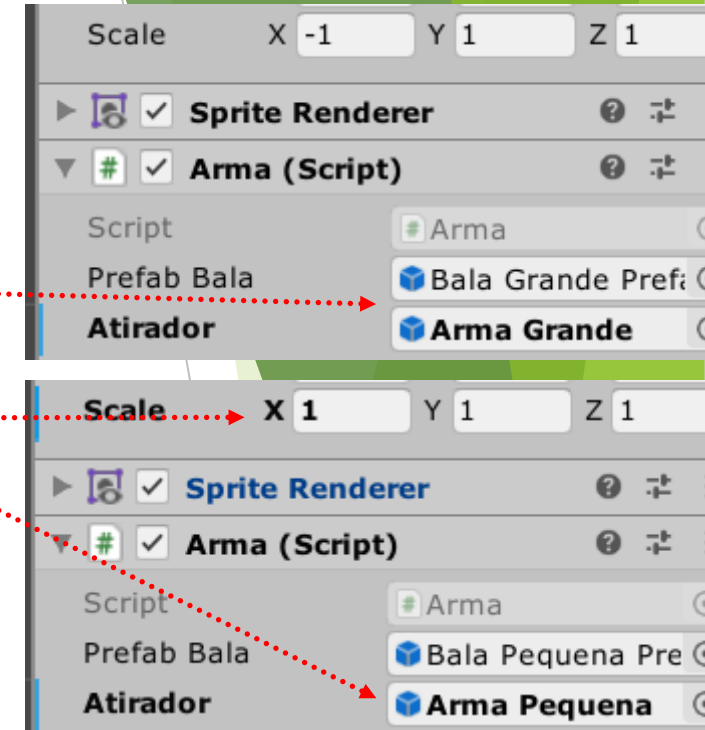
Prática 3 - Controlador do Inimigo

- ▶ Arraste a arma para a pasta Prefabs da janela Project, pois a faremos ser, também, um prefab. Observe que a alteração do nome do Bala Prefab já se refletiu no script Arma desse objeto.
- ▶ Nesse novo prefab, selecione “Bala Pequena Prefab” como o valor da propriedade Prefab Bala. Selecione **None** como o valor da propriedade Atirador, pois usaremos o Inspector de cada objeto criado a partir desse modelo para definir o atirador.
- ▶ Saia da edição desse Prefab, e remova a Arma original da cena. Mude o nome do Prefab para Arma Grande.
- ▶ Usando [Ctrl-D], duplique esse prefab e o chame de Arma Pequena. Note que não colocamos a palavra Prefab nesses objetos. Obviamente, por estarem na pasta Prefabs, se espera que sejam prefabs. Você decide se coloca ou não essa palavra.
- ▶ Vamos editar o prefab Arma Pequena, selecionando-o na pasta e pressionando o botão [Open Prefab]. Mudaremos o sprite a ele associado, colocando o sprite da pequena arma que vemos na seleção de sprites (clique no ícone do círculo para abrir o seletor).
- ▶ Note que o PontoDeTiro ficou fora da boca do cano da arma, pois o sprite é menor que o anterior.
- ▶ Ajuste a posição do PontoDeTiro para que fique logo na saída da boca do cano da arma.
- ▶ Saia da edição desse prefab e vamos testar o que fizemos.



Prática 3 - Controlador do Inimigo

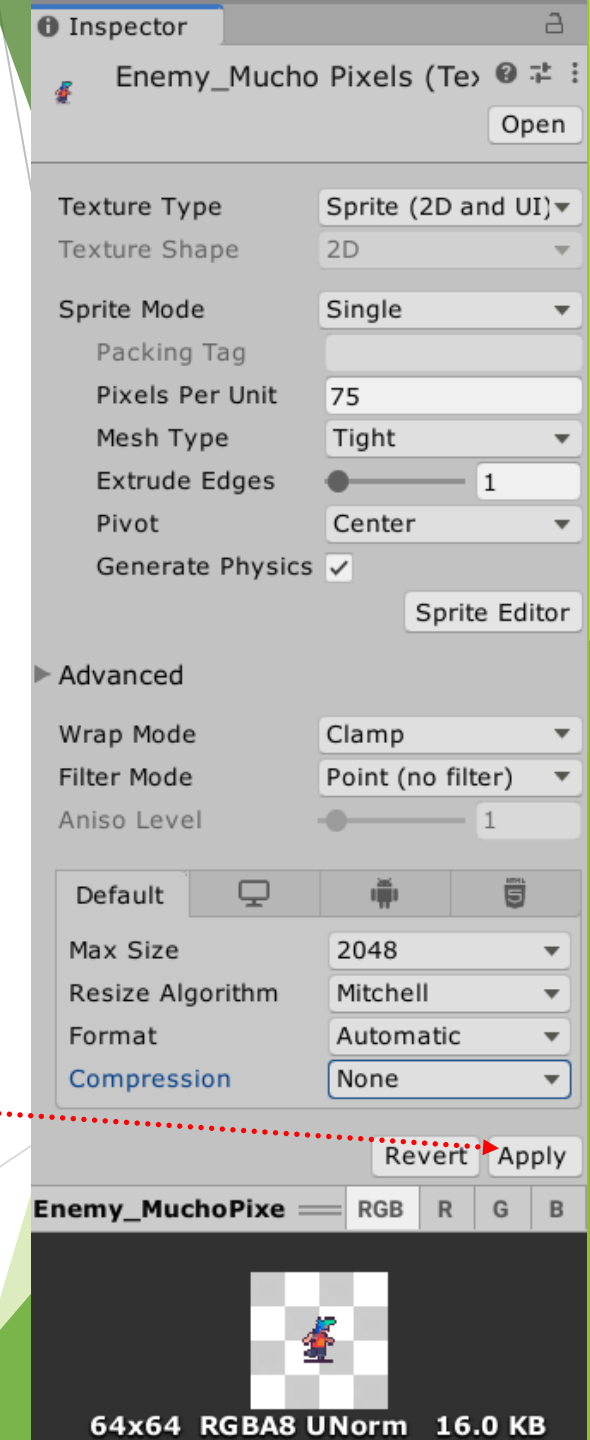
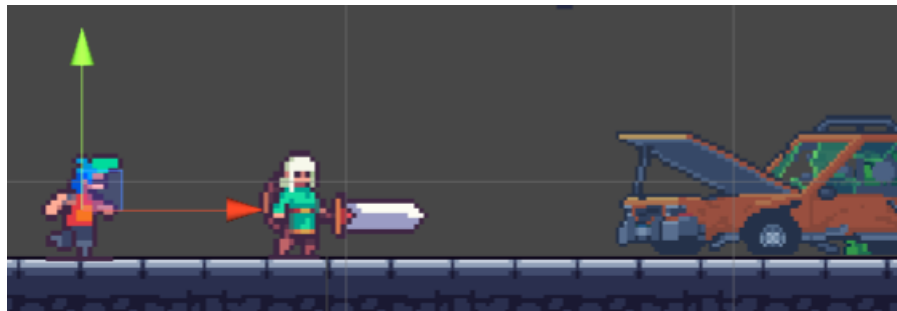
- ▶ Crie na cena uma instância do prefab da Arma Grande e uma instância do prefab da Arma Pequena, arrastando os prefabs para a cena.
- ▶ Associe o próprio objeto criado com a propriedade Atirador de cada um deles.
- ▶ No objeto da Arma Pequena, mude a propriedade Scale.X para 1, de forma que essa arma aponte para a direita.
- ▶ Executando e pressionando a tecla [Alt], disparamos 3 tiros de cada arma, com Balas de aparência e velocidades diferentes.



- ▶ Ajuste os tamanhos das balas caso ache necessário, editando seus prefabs.

Prática 3 - Controlador do Inimigo

- ▶ Assim, já temos nossos prefabs preparados para serem usados.
- ▶ Essa abordagem é muito útil, pois lhe permite criar inimigos, itens coletáveis, itens de interface, bombas, partículas como explosões, NPC (Non-player Character) ou “personagens não controlados pelo jogador”.
- ▶ Quando perceber que um objeto está sendo usado muitas vezes, é bom convertê-lo num prefab e criá-lo por programação, como estamos fazendo neste estudo.
- ▶ Agora vamos estudar o conceito de Corrotinas. Corrotinas são métodos, como as que vimos anteriormente, mas que nos permitem executar uma série de passos por intervalos, intercalando ações e tempos de espera.
- ▶ Para usar esse conceito, vamos primeiramente importar, em nosso jogo, um Asset para o oponente. Esse asset é a imagem Enemy_MuchoPixels fornecido no material da prática.
- ▶ Após importá-lo para a pasta Sprites da janela Project, configure-o como sempre: **Sprite Mode** = Single, **Pixels per Unit** = 75, **Filter Mode** = Point (no filter), **Compression** = None. Aplique as alterações pressionando o botão [Apply].
- ▶ Coloque-o na cena, à esquerda do Player e o chame de Oponente.



Prática 3 - Controlador do Inimigo

- Agora, criaremos um script para controlar o oponente; esse script se chamará PatrulhaDoOponente. A ideia geral desse script é que o oponente se mova para a direita até um ponto máximo, dê a volta e se mova para a esquerda até um ponto máximo, e repita esses movimentos infinitamente. Associe o script ao objeto Oponente na cena. abra sua edição pelo Visual Studio e digite o trecho abaixo:

```
public class PatrulhaDoOponente : MonoBehaviour
{
    public float velocidade = 1f;
    public float minX, maxX;
    public float tempoDeEspera = 2f;
    private GameObject _meta;

    private void AtualizarMeta()
    {
        // se é a primeira vez que chama este método, cria a meta na esquerda
        if (_meta == null)
        {
            _meta = new GameObject("Alvo");
            _meta.transform.position = new Vector2(minX, transform.position.y);
            return;
        }

        // se estamos na esquerda, trocamos a meta para a direita
```

Prática 3 - Controlador do Inimigo

- ▶ Continue digitando o método AtualizarMeta() e, depois, chame esse método no tratador de evento Start():

```
// se estamos na esquerda, trocamos a meta para a direita
if (_meta.transform.position.x == minX)
{
    _meta.transform.position = new Vector2(maxX, transform.position.y);
    transform.localScale = new Vector3(1,1,1);
}
// se estamos na direita, trocamos a meta para a esquerda
else
{
    if (_meta.transform.position.x == maxX)
    {
        _meta.transform.position = new Vector2(minX, transform.position.y);
        transform.localScale = new Vector3(-1,1,1);
    }
}

void Start()
{
    AtualizarMeta();
}
```

Prática 3 - Controlador do Inimigo

- ▶ Nesse script, primeiramente declaramos quatro variáveis públicas; a velocidade do oponente enquanto se move, a coordenada do lado esquerdo mínimo desse movimento, a coordenada do lado direito (máxima) desse movimento, ambas no eixo X (horizontal) e o tempo que o oponente aguardará quando chega a uma dessas metas.
- ▶ Em seguida, declaramos uma variável privativa chamada `_meta`, que indicará o local do movimento máximo do Oponente na direção em que ele se move. Essa variável não é iniciada ainda, ficará com null, portanto. Ela será iniciada na primeira vez que esse método for chamado (pelo tratador de evento `Start()`).
- ▶ No método `AtualizarMeta()`, primeiramente vemos se `_meta` vale null. Se isso acontece, significa que é a primeira vez que esse método é executado pois, logo em seguida, essa variável será instanciada e receberá um objeto criado nesse exato momento, em tempo de jogo, para armazenar os limites do movimento do oponente. Esse método será chamado em outros momentos do jogo e, a partir dessa primeira chamada, essa variável não mais valerá null durante a execução desse jogo. O objeto criado e armazenado em `_meta` será invisível durante o jogo, pois não tem `Render`.
- ▶ O fluxo retorna para o local de chamada após criar `_meta`, para não executar os comandos seguintes.
- ▶ Os comandos `If` e `else` seguintes verificam se, durante seu movimento, o oponente chegou à meta (ao limite horizontal de seu movimento). Caso tenha chegado, sua escala em X ficará o oposto da atual (-1 se valer 1, 1 se valer -1) de forma que o oponente passa a apontar para o outro lado.

Prática 3 - Controlador do Inimigo

- ▶ Em seguida, criaremos a corrotina, que basicamente moverá o oponente até a meta atual, a partir do local onde ele esteja. Na primeira vez, ele se moverá para a esquerda.
- ▶ A corrotina é um método que retorna uma interface IEnumerator; iniciamos sua codificação a seguir:

```
IEnumerator PatrulhaAteMeta()  
{  
    // Corrotina para mover o oponente  
    while (Vector2.Distance(transform.position, _meta.transform.position) > 0.05f)  
    {  
        Vector2 direcao = _meta.transform.position - transform.position;  
        float direcaoX = direcao.x;  
        transform.Translate(direcao.normalized * velocidade * Time.deltaTime);  
        // IMPORTANTE:  
        yield return null;  
    }  
}
```

// neste ponto, encontramos a meta, e vamos ajustar a posição do oponente para a da meta:

- ▶ Como dissemos antes, uma corrotina é um tipo especial de método que nos permite executar tarefas em etapas, estabelecer tempos entre as etapas.

Prática 3 - Controlador do Inimigo

- ▶ No código anterior, o comando while é repetido enquanto a distância entre a posição atual do objeto associado ao script até a posição da meta de movimento fique menor que 0.05f. O método Distance é usado para calcular a distância entre os dois pontos.
- ▶ Se essa distância for maior que 0.05f, é calculado o vetor de direção do movimento e se usa o método Translate para mover o oponente na direção desejada, por uma pequena fração de movimento, de acordo com a velocidade e o deltaTime.
- ▶ yield return null faz com que se saia do while e se retorne imediatamente ao início da corrotina, de forma que novamente se verifica a distância do oponente à meta.
- ▶ Quando a condição do while for falsa, teremos de atualizar a posição do oponente, fazê-lo esperar em sua patrulha e reiniciar a corrotina. Isso é feito abaixo:

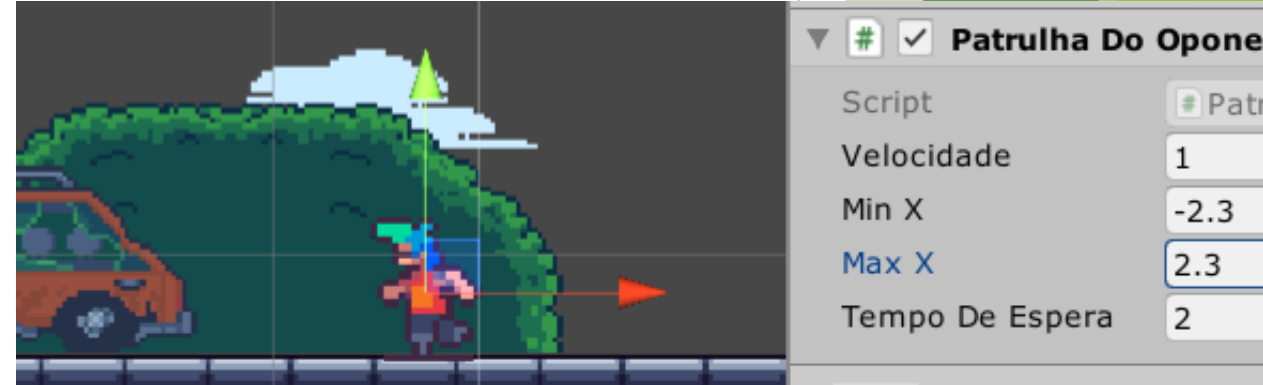
```
// neste ponto, encontramos a meta, e vamos ajustar a posição do oponente para a da meta:
Debug.Log("Meta encontrada!");
transform.position = new Vector2(_meta.transform.position.x, transform.position.y);

// e esperamos pelo tempo definido pela variável tempoDeEspera:
Debug.Log($"Esperando por {tempoDeEspera} segundos");
yield return new WaitForSeconds(tempoDeEspera);

// tendo esperado, vamos restaurar o comportamento de patrulha
Debug.Log("Esperou tempo bastante, vamos atualizar a meta e andar de novo");
AtualizarMeta();
StartCoroutine("PatrulhaAteMeta");
```

Prática 3 - Controlador do Inimigo

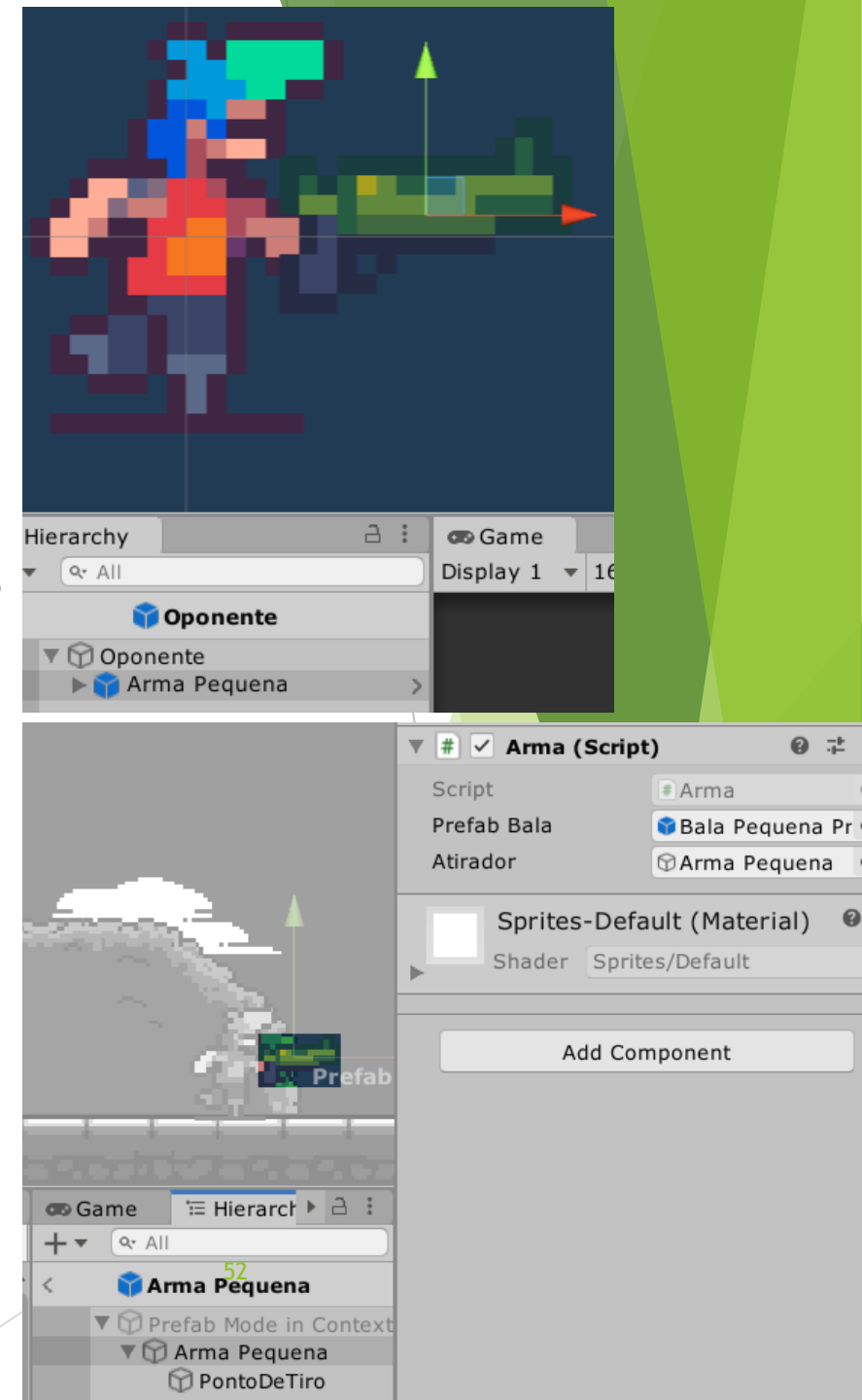
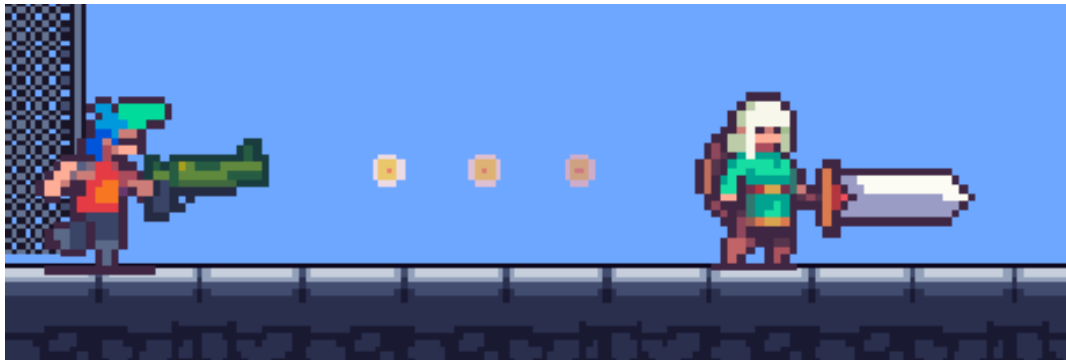
- ▶ `yield return new WaitForSeconds(tempoDeEspera)` diz à corrotina para ela parar sua execução e retornar ao fluxo quando o tempo de espera passar.
- ▶ Caso o game loop execute a corrotina várias vezes, o uso de `yield` fará com que o fluxo de execução retorne sempre após esse comando, não do início.
- ▶ Esse tipo de método nos permite fazer diversos comportamentos complexos aos nossos objetos de jogo, principalmente os personagens não controlados pelo jogador (NPC).
- ▶ Atribua os valores de `minX` e `maxX` no Inspector do objeto de cena Oponente. Você pode mover o oponente usando a Move Tool até cada posição extrema do chão que achar conveniente e usar os valores de `Position.X` para `minX` e `maxX`, como vemos na figura ao lado, em que `minX` vale -2.3 e `maxX` vale 2.3:
- ▶ No tratador de evento `Start()`, chame a corrotina pela primeira vez, de forma que, a partir daí, ela passa a se chamar continuamente.



```
void Start()
{
    AtualizarMeta();
    StartCoroutine("PatrulhaAteMeta");
}
```

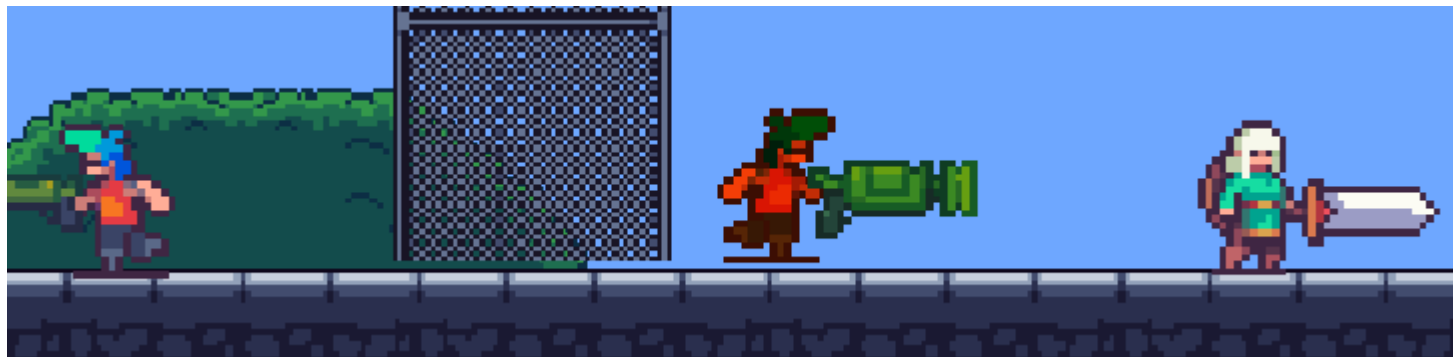
Prática 3 - Controlador do Inimigo

- ▶ Finalmente, remova as armas da cena, deixando-as apenas como prefabs na janela Project.
- ▶ Faça com que o Oponente da Hierarquia seja transformado em mais um Prefab, de forma que possamos criar mais de um exemplar de inimigos na cena, se assim desejarmos.
- ▶ Faça reset no transform desse prefab e abra-o, pressionando o botão [Open Prefab] do Inspector. Na janela de edição de Prefabs, arraste o prefab da Arma Pequena para junto à mão do Prefab Oponente (mude Scale.X para 1, se necessário), como vemos na figura ao lado:
- ▶ Veja que estamos usando um prefab aninhado em outro. Se você acessar o prefab aninhado no Oponente da cena e associar, no script Arma, o próprio objeto Arma Pequena na Hierarquia, ele passará a atirar, também, enquanto o Oponente anda na cena.



Prática 3 - Controlador do Inimigo

- ▶ Você pode criar um novo exemplar do Prefab Oponente, agora com a arma grande, e ajustá-lo da mesma maneira para que ele também atire.
- ▶ Chame-o de Oponente Chefao.
- ▶ Poderá mudar a propriedade Color (no Sprite Renderer) do Oponente com a arma maior, para dar-lhe uma aparência diferente e mais ameaçadora, por exemplo.
- ▶ Assim, poderá ter dois personagens diferentes em momentos distintos da cena.



- ▶ No entanto, as balas vão sempre para a direita. Revise o código e as configurações dos objetos e pontos de tiro para verificar o que pode ser feito para que isso não aconteça, ou seja, para que as balas sejam disparadas na direção em que a arma e o oponente apontam.
- ▶ Diminua a escala da arma do Prefab Arma Pequena, pois ela ainda está muito grande em proporção ao Oponente e, um pouco menor, ela terá uma aparência melhor em nosso estudo seguinte, sobre Animação.